Decoder Models and Encoder-Decoder Systems

Language Models for Law and Social Science – Week 8

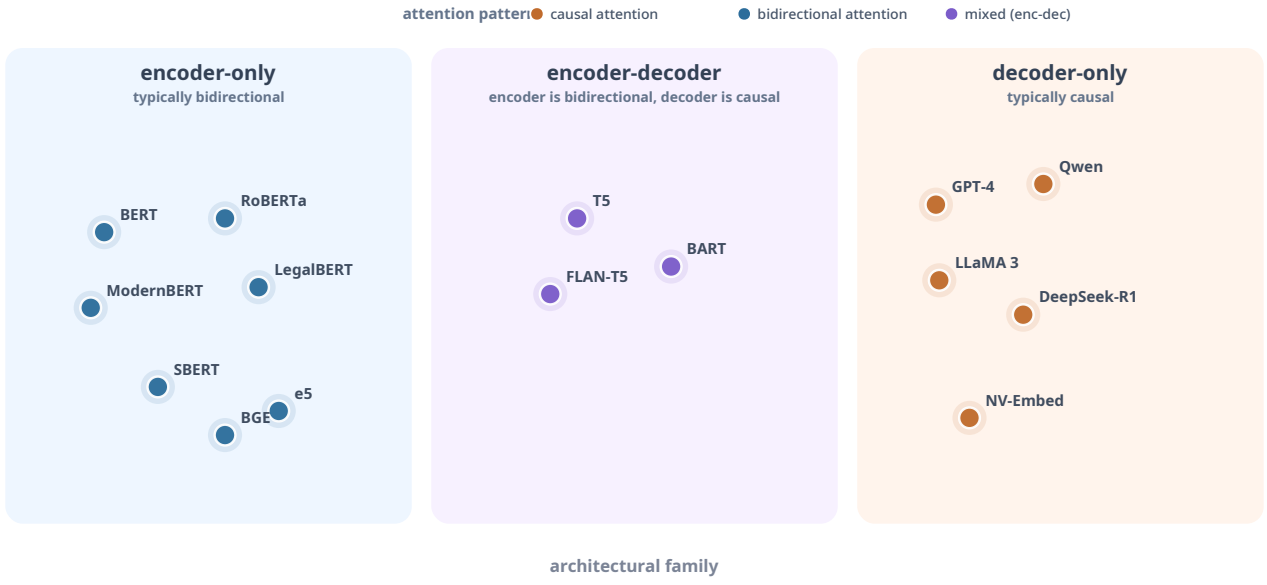
David Zollikofer

ETH Zürich

April 13, 2026

Outline

Where We Are



Last week: encoders (BERT and friends). Today: the other two boxes.

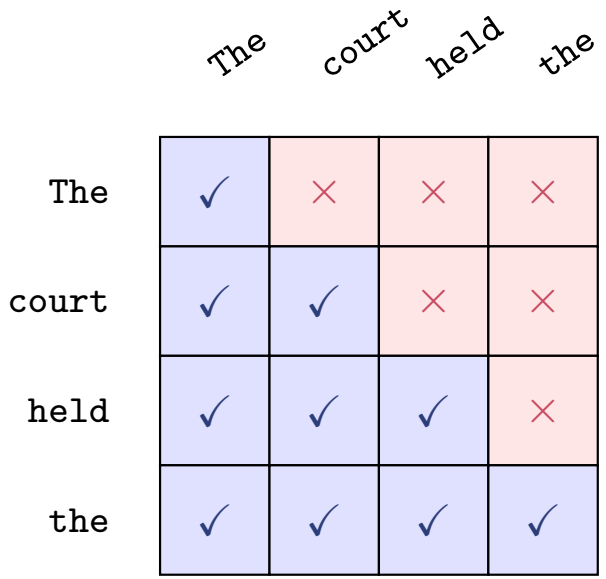
Recap: What We Had with BERT

For a sequence of length n, BERT gives us:

- n positions, each attending to **all** n positions (bidirectional)
- Rich contextual representations at every position
- But the training signal comes from only $\sim$15% of tokens (masked language modeling)

The Causal Mask (Vaswani et al., 2017)

Restrict each position to only attend to **itself and the past**:



Position 1 sees only itself.
Position n sees everything (like BERT).

Implementation: add $-\infty$ before the softmax.

$$a_{ij} = \text{softmax}_j\left(\frac{q_i^\top k_j + M_{ij}}{\sqrt{d_k}}\right)$$

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases}$$

Same Transformer block as BERT. Same Q, K, V projections, same FFN, same residuals.

The only architectural difference is this mask. Everything else – multi-head attention, residual connections, layer norm, FFN is identical.

All n Positions in One Forward Pass

The causal mask lets us train on **every position simultaneously**. Feed the full sequence in one forward pass – the mask prevents each position from seeing the future:

Position	Sees	Predicts
1	`[BOS]`	`The`
2	`[BOS] The`	`court`
3	`[BOS] The court`	`upheld`
4	`[BOS] The court upheld`	`the`
5	`[BOS] The court upheld the`	`appeal`

This is **teacher forcing**: we feed ground-truth tokens, not the model's own predictions. One sequence of length n yields n loss terms:
$\mathcal{L} = -\sum_{t=1}^{n} \log p(x_t \mid x_{<t})$

Compare: BERT's MLM masks $\sim$15% of tokens and only those produce a loss. Here, **every** position contributes.

The Tradeoff

	Encoder (BERT)	**Decoder (GPT)**
Context per position	Full: all n tokens	Partial: only past tokens
Training signals	$\sim 0.15n$ (masked tokens)	$\sim n$ (every position)
Training parallelism	Fully parallel	Fully parallel (teacher forcing)
Inference	One forward pass	Sequential: one token at a time

Early positions in a decoder have impoverished context compared to BERT. But every position contributes to the loss, and training is *closer* to inference than in BERT: no future tokens are visible. Still, training uses the true prefix, while generation uses the model's own prefix.

GPT-1 → GPT-2 → GPT-3 (Radford et al., 2018; 2019; Brown et al., 2020)

GPT-1 (2018)

117M params.
Pretrained decoder, then **fine-tuned** per task.
Same paradigm as BERT, just with a causal model.

GPT-2 (2019)

1.5B params.
Coherent long-form generation **without fine-tuning**. First serious evidence that scale alone unlocks new behavior.

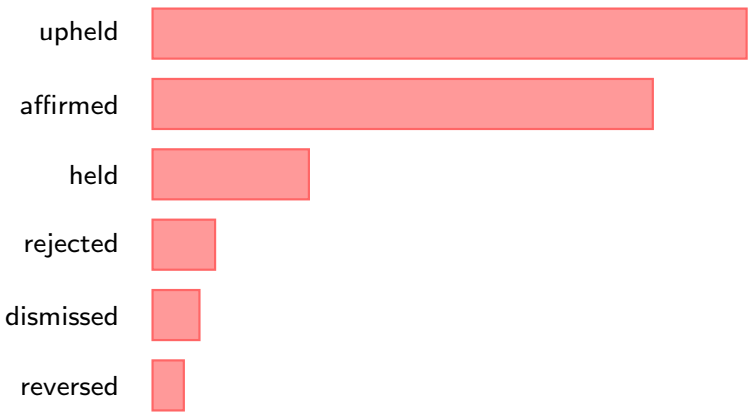
GPT-3 (2020)

175B params.
Prompting becomes a usable interface.
Zero-shot, one-shot, few-shot – no gradient updates needed.

Text Generation

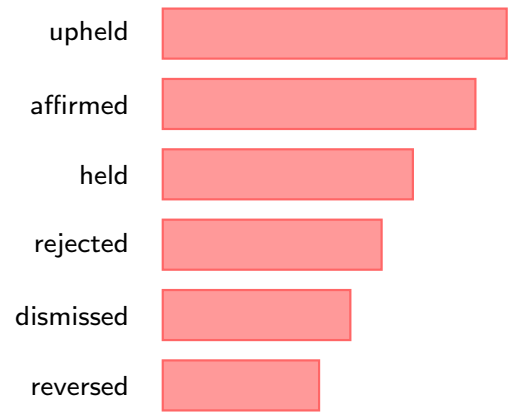
At inference, generation is sequential. The model outputs $p(x_{t+1} \mid x_{\leq t})$. Temperature τ rescales logits before softmax: $p_i \propto \exp(z_i/\tau)$.

Low temperature ($\tau = 0.3$)



Peaked: top tokens dominate

High temperature ($\tau = 1.5$)



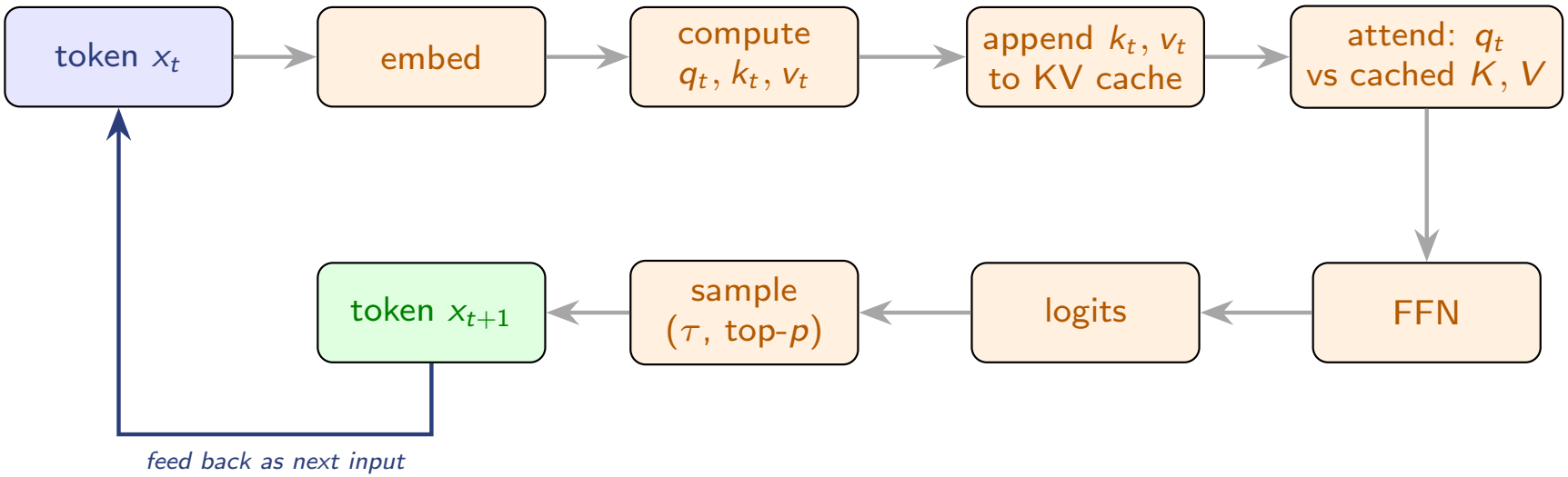
Flatter: unlikely tokens get more mass

Other strategies: **greedy** (argmax), **top-k** (sample from k best), **top-p** / nucleus (adaptive cutoff).

Setting a **random seed** can make sampling deterministic.*

*Hardware non-determinism may cause small differences. Not all providers expose a seed parameter.

How Decoder Inference Works



Each iteration produces **one token**. The loop repeats until a stop condition (max length, EOS token).

The bottleneck: at each step, attention must touch the entire KV cache. Longer sequences = slower steps + more memory.

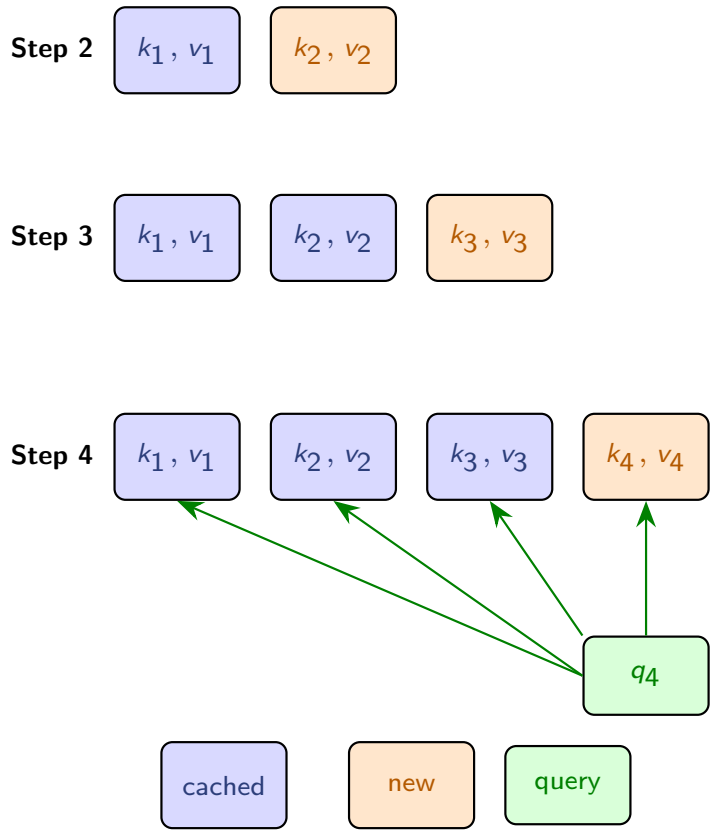
Making Decoders Fast: KV Caching

At step t, attention computes $q_t^\top k_j$ for all $j \leq t$.

Without cache: recompute K, V for all tokens at every step. Total cost for n tokens: $O(n^2)$.

With cache: store K, V from previous steps. At step t, only compute k_t, v_t for the new token and append.

Why not cache Q? The query q_t is only needed at step t – it compares the *current* token against all previous ones. Previous queries $(q_1, \ldots, q_{t-1})$ are never reused. Keys and values, by contrast, are needed at *every future step*.



The KV cache grows linearly with sequence length – this is why longer context windows need more GPU memory.

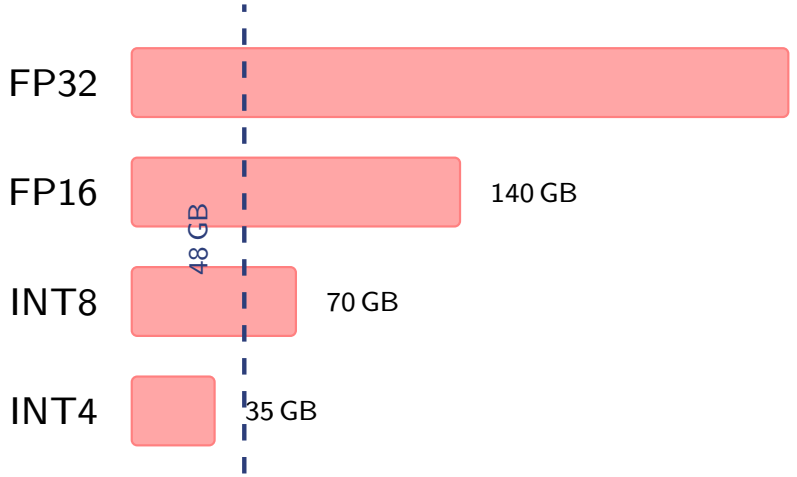
Making Decoders Fast: Quantization

Model weights are stored as floating-point numbers. For inference, full precision is often unnecessary.

Idea: round weights to lower precision after training.

Format	Bits/param	70B model
FP32	32	280 GB
FP16 / BF16	16	140 GB
INT8	8	70 GB
INT4	4	35 GB

INT4 can shrink a 70B model enough that it may run on a 48 GB GPU. In practice, this still depends on overhead, KV cache, and the exact setup.



Tools: `bitsandbytes`, GPTQ, AWQ, GGUF (llama.cpp)

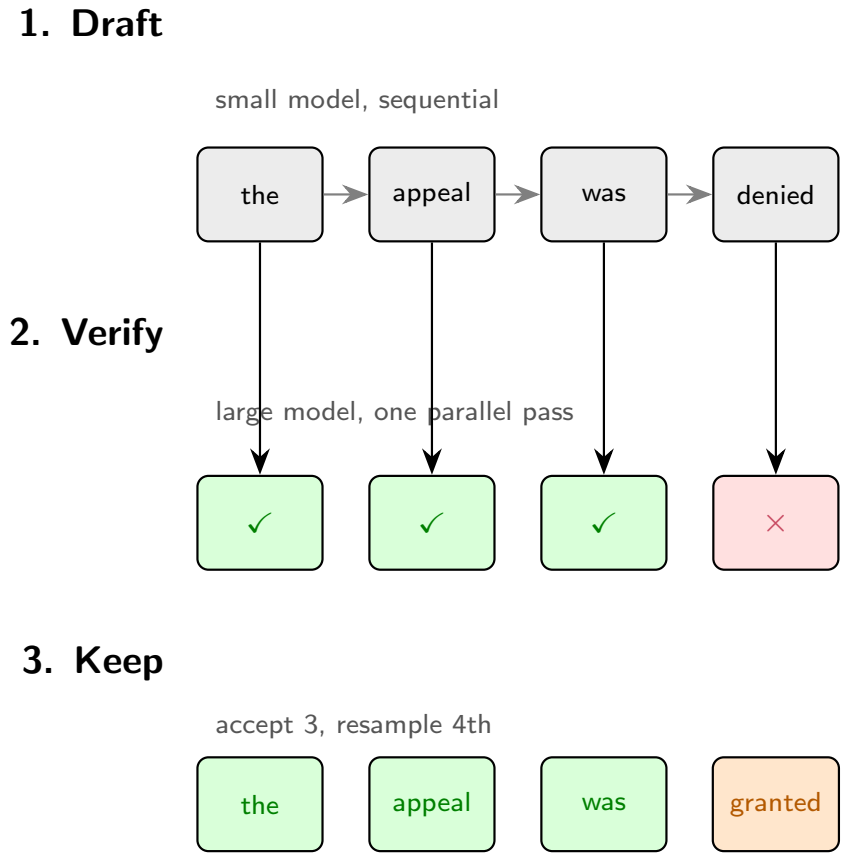
Making Decoders Fast: Speculative Decoding

Normal generation: one forward pass per token through the large model. Slow.

Idea: use a small, fast **draft model** to propose k tokens. Then let the large model **verify all** k **in one forward pass**.

Why does this work? The large model can check k drafted tokens in one parallel pass, similar to training with teacher forcing. Checking 4 tokens costs roughly the same as generating 1.

Accept the longest agreed prefix; **resample** at the first disagreement. Typical speedup: 2–3× with identical output distribution.



How Decoder Models Are Trained

Data: large web corpora (Common Crawl, Wikipedia, books, code) – billions to trillions of tokens.

Objective: next-token prediction. Minimize cross-entropy:

$$L = -\frac{1}{n}\sum_{t=1}^{n} \log p(x_t \mid x_{<t})$$

What is measured:

- **Test loss** – cross-entropy on unseen text
- **Perplexity** – $\exp(L)$
- Downstream benchmarks (MMLU, etc.)

Typical training recipe:

1. Collect and filter a large text corpus
2. Tokenize (BPE / SentencePiece)
3. Train with AdamW, cosine LR schedule
4. Thousands of GPU-hours to GPU-years

The next slide shows test loss as a function of scale – the same quantity optimized during training, measured on held-out data.

Scaling Laws (Kaplan et al., 2020)

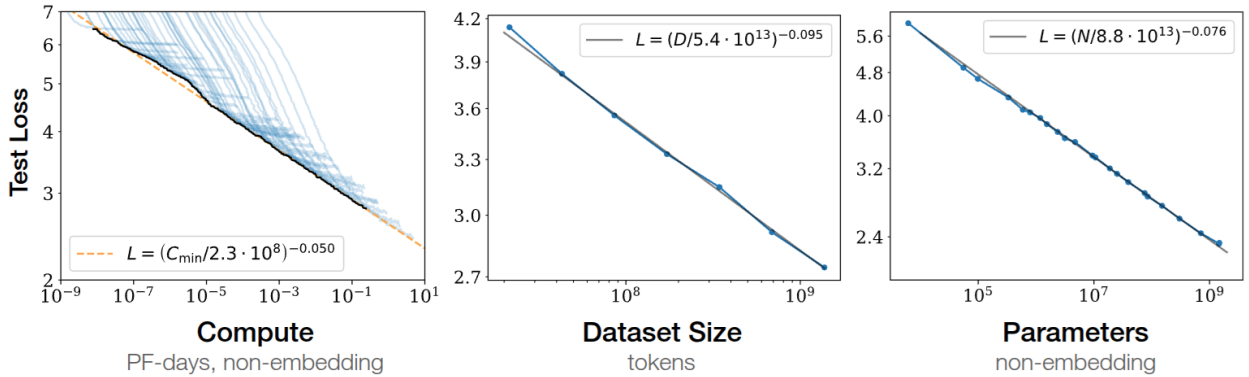


Figure 1 Language modeling performance improves smoothly as we increase the model size, datasetset size, and amount of compute[2] used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Test loss decreases as a **power law** in compute, data, and parameters: $L \propto C^{-0.050}$, $L \propto D^{-0.095}$, $L \propto N^{-0.076}$. Straight lines on log-log axes = power law. Each blue line is a different model size. All three factors must be scaled together.

Zero-Shot, One-Shot, Few-Shot (Brown et al., 2020)

The task is specified **in the prompt**, not in updated weights:

Zero-shot

```
Classify as
supportive or
critical:

"The ruling
undermines public
trust."
```

→ **critical**

One-shot

```
"The reform improves
transparency." →
supportive

"The ruling
undermines public
trust."
```

→ **critical**

Few-shot

```
"Improves
transparency." →
supp.
"Weakens
accountability." →
crit.
"Protects tenants."
→ supp.

"The ruling
undermines public
trust."
```

→ **critical**

No gradient updates. The examples shape the context the model conditions on.

Base Models vs. Instruction-Tuned Models

Everything so far describes **base models**: trained only to predict the next token. A base model completes text – it does not "follow instructions."

Base model (e.g. GPT-3)

- Trained on: next-token prediction
- Behavior: continues the text
- Prompt: requires careful formatting
- Few-shot examples help a lot

Instruction-tuned (e.g. ChatGPT)

- Additional training to follow instructions
- Behavior: answers, follows requests
- Prompt: natural language works
- Much easier to use in practice

How instruction tuning works (SFT, RLHF) is a later topic. For now: ChatGPT, Claude, etc. are *not* raw base models.

The Encoder-Decoder Idea

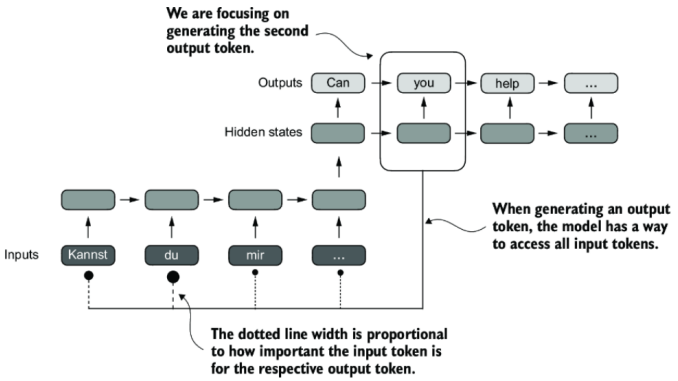


Figure 3.5 Using an attention mechanism, the text-generating decoder part of the network can access all input tokens selectively. This means that some input tokens are more important than others for generating a given output token. The importance is determined by the attention weights, which we will compute later. Note that this figure shows the general idea behind attention and does not depict the exact implementation of the Bahdanau mechanism, which is an RNN method outside this book's scope.

Figure adapted from Raschka (2024)

A decoder reads a prefix and continues it. Sometimes the input and output are fundamentally different things (e.g., German $\rightarrow$ English).

Encoder-decoder models split the job:

- **Encoder:** reads the full input bidirectionally, produces states $h_1, \ldots, h_n$
- **Decoder:** generates output autoregressively, conditioned on those states

Natural fit for translation, summarization, question answering, any task where input and output have different structure.

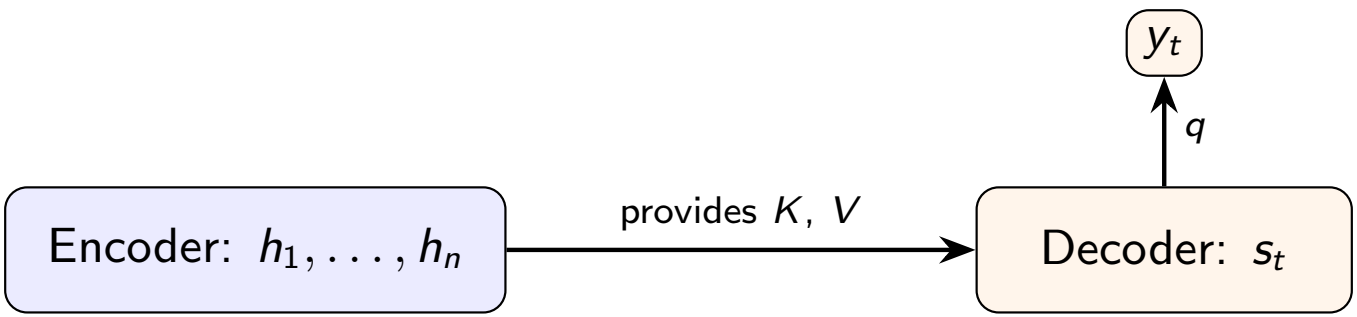
Cross-Attention

Each decoder block has two attention mechanisms:

1. **Causal self-attention** over previously generated tokens (same as GPT)
2. **Cross-attention** over the encoder states

Cross-attention: queries come from the decoder, keys and values from the encoder.

$$\text{CrossAttn}(q^{\text{dec}},\ K^{\text{enc}},\ V^{\text{enc}}) = \text{softmax}\left(\frac{q^{\text{dec}}(K^{\text{enc}})^\top}{\sqrt{d_k}}\right) V^{\text{enc}}$$



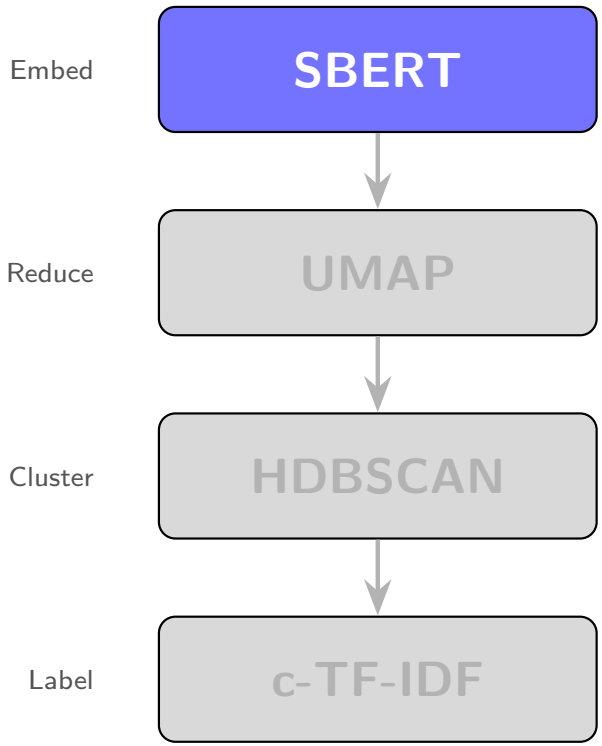
This is how the decoder conditions *every* generated token on the full input.

T5 and BART (Raffel et al., 2020; Lewis et al., 2020)

	T5	BART
Core idea	Every task is text → text	Corrupt text, train to reconstruct it
Encoder input	Text with sentinel tokens replacing spans: `The <X> ruling <Y>`	Corrupted text (tokens masked, deleted, or shuffled)
Decoder target	Only the missing spans: `<X> court's <Y> was`	Full original text, reconstructed from scratch
Decoder cost	Short: generates only the corrupted spans	Full: generates the entire input sequence
Loss	Cross-entropy on decoder output, same as any autoregressive model: $-\sum_t \log p(y_t \mid y_{<t}, \text{encoder states})$	

Both use a bidirectional encoder + causal decoder. Largely superseded by decoder-only models at large scale, but still widely used for structured generation tasks.

BERTopic Pipeline (Grootendorst, 2022)



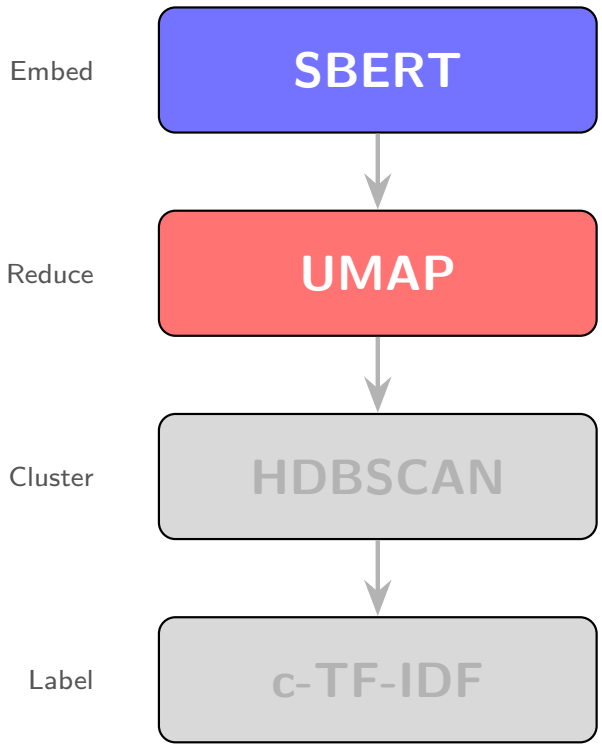
Not a language model – a **topic-modeling pipeline** on top of embeddings.

1. Embed each document with a sentence encoder (e.g. all-MiniLM-L6-v2).

Each document becomes a dense vector in $\mathbb{R}^{384}$. Semantically similar documents are nearby in this space, even if they share no words.

Choice of embedding model matters: a domain-adapted model (e.g. legal, biomedical) can improve downstream cluster quality.

BERTopic Pipeline (Grootendorst, 2022)



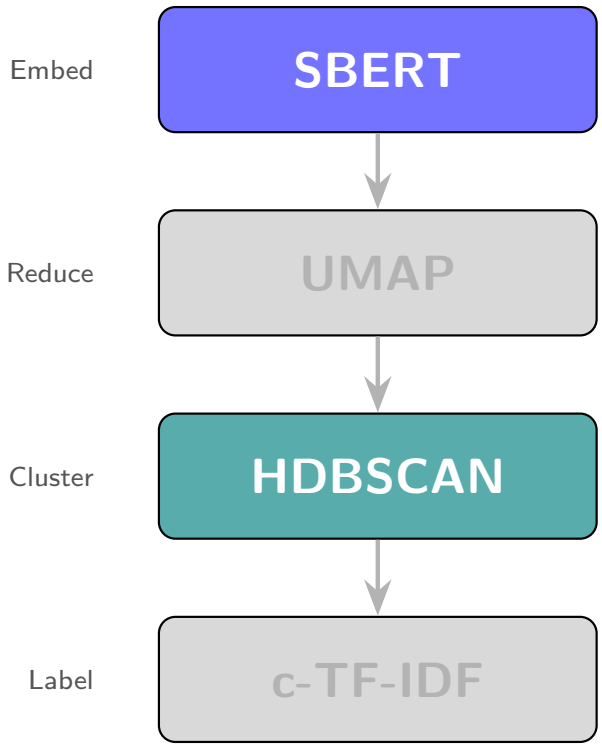
Not a language model – a **topic-modeling pipeline** on top of embeddings.

2. UMAP – Uniform Manifold Approximation and Projection.

Reduces ∼384 dimensions to ∼5. Preserves *local* neighborhood structure: points that were nearby in high-dimensional space stay nearby.

Why not cluster directly in $\mathbb{R}^{384}$? In high dimensions, distances between points converge – everything looks equally far apart (curse of dimensionality). UMAP makes the cluster structure visible.

BERTopic Pipeline (Grootendorst, 2022)



Not a language model – a **topic-modeling pipeline** on top of embeddings.

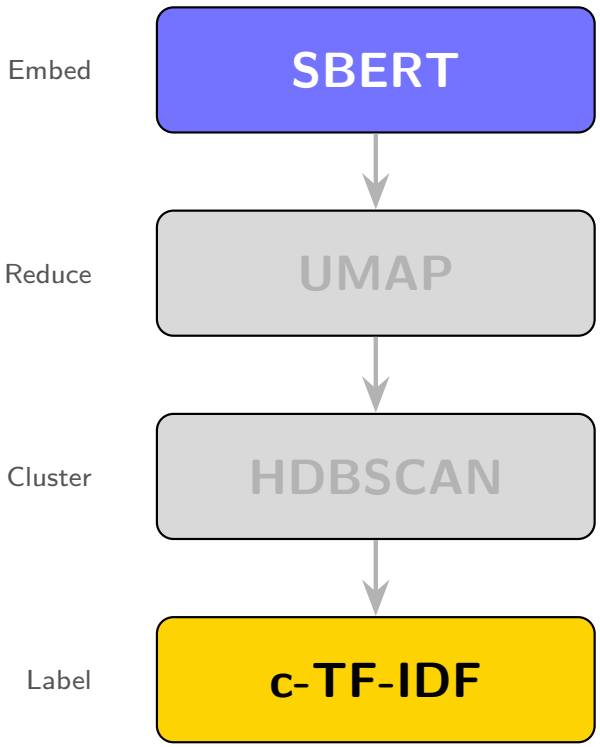
3. HDBSCAN – Hierarchical Density-Based Spatial Clustering.

Finds clusters as dense regions in the reduced space. Key properties:

- Does *not* require specifying the number of clusters
- Handles clusters of different sizes and shapes
- Points in sparse regions become outliers (topic -1)

Contrast with k-means: no forced assignment, no spherical assumption.

BERTopic Pipeline (Grootendorst, 2022)



Not a language model – a **topic-modeling pipeline** on top of embeddings.

4. c-TF-IDF – class-based TF-IDF.

Concatenate all documents in one cluster into a single "class document." Then compute a modified TF-IDF:

$$\text{c-TF-IDF}_i = \frac{t_i}{w_i} \cdot \log\left(1 + \frac{A}{\text{tf}_i}\right)$$

where t_i is the frequency in this cluster, w_i the total words in the cluster, A the average words per cluster, and tf_i the frequency across all clusters.

Top-scoring words become the topic label.

BERTopic in Practice

```python
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

topic_model = BERTopic(
    embedding_model=embedding_model,
    min_topic_size=15,
    nr_topics="auto",
)
topics, probs = topic_model.fit_transform(documents)

# Top words per topic
topic_model.get_topic_info()

# Reduce number of topics post-hoc
topic_model.reduce_topics(documents, nr_topics=20)

# Interactive visualization
topic_model.visualize_topics()
```

BERTopic vs. LDA

	BERTopic	**LDA**
Input representation	Dense embeddings	Bag-of-words
Short texts	Works well	Noisy (few words per doc)
Paraphrases	Cluster together	End up in different topics
Number of topics	Determined by HDBSCAN; adjustable post-hoc	Must be set in advance
Compute	GPU helpful for large corpora	CPU-only, fast

LDA remains reasonable when corpora are large, compute is limited, or you need a probabilistic generative model. BERTopic is stronger when semantic similarity matters more than word co-occurrence.

Application: Wage Gap Estimation (Vafa, Athey, and Blei, 2025)

Problem: estimating the unexplained gender wage gap. Traditional methods use coarse career summaries (years of experience, broad occupation) – omitting important factors.

Approach: train a foundation model (CAREER) on 24M resumes to predict next occupation – a next-token objective over career histories.

Fine-tune on wage prediction (PSID survey data). Learned representations capture richer career history than hand-crafted variables.

Key findings:

- Career history explains more of the wage gap than standard econometric models
- Predictions outperform standard models by 10–15%
- Standard fine-tuning can introduce **omitted variable bias**
- They develop debiased fine-tuning to address this

A concrete example of pretrain $\rightarrow$ fine-tune for social science *measurement*, not just prediction.

Prompting as Measurement

Key takeaways from many CSS papers for your own research (e.g. Ziems et al., 2024).

When you use an LLM to code or classify data, the prompt is part of the **measurement instrument**.

Reliability	Even at temperature 0, paraphrasing the same instruction changes label distributions.
Validity	LLMs use colloquial meanings, not your codebook's definitions. In emotion classification, models systematically confuse *joy* and *love* because they overlap in everyday language, even though your coding scheme treats them as distinct.
Bias	Label ordering, example selection, and framing affect output distributions. Models have their own priors about politically charged topics.

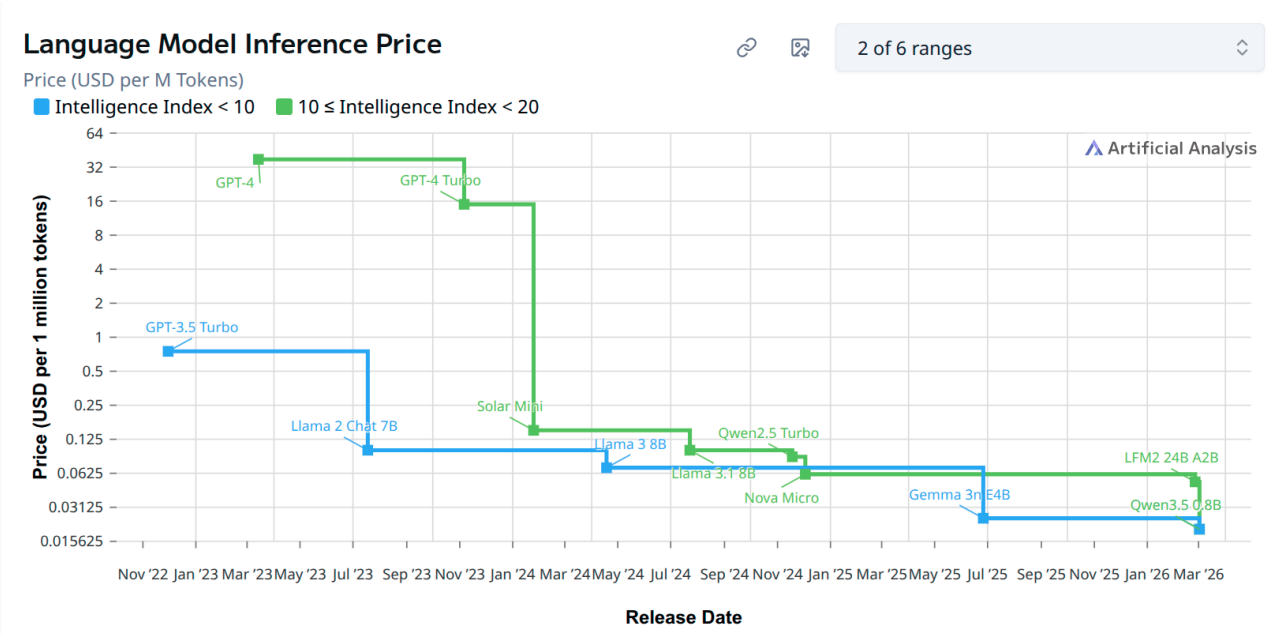
Report your model, prompt template, temperature, number of runs, and inter-run agreement.

Limitations of Decoder Models

- **Hallucination:** confident generation of false statements.
- **Prompt sensitivity:** small wording changes ("classify" vs. "categorize", label order) can shift results substantially.
- **Sampling variance:** with temperature > 0, the same prompt produces different outputs on each run. This is a source of measurement error.
- **Cost:** generating 60 tokens with a 70B model takes orders of magnitude more compute than computing one BERT embedding.

These are not engineering inconveniences – for empirical research, they are threats to inference that need to be addressed in the research design.

Inference Cost Is Dropping Fast



Note: y-axis is **logarithmic**: The price drop is orders of magnitude, not linear. Take specific numbers with a grain of salt (benchmarks, pricing tiers, and capabilities vary), but the overall trend is clear: inference is getting dramatically cheaper and more capable over time.

Source: artificialanalysis.ai/trends

Recap

- The decoder is the same Transformer block as the encoder, with a causal mask
- That mask enables dense training (all n positions) but makes generation sequential
- Scale turned prompting into a usable research tool (few-shot learning)
- BERTopic bridges modern embeddings and classical topic modeling

References

- Brown, T. et al. (2020). Language models are few-shot learners. *NeurIPS*.
- Grootendorst, M. (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure.
- Hoffmann, J. et al. (2022). Training compute-optimal large language models. *NeurIPS*.
- Kaplan, J. et al. (2020). Scaling laws for neural language models.
- Lewis, M. et al. (2020). BART: Denoising sequence-to-sequence pre-training. *ACL*.
- Radford, A. et al. (2018). Improving language understanding by generative pre-training.
- Radford, A. et al. (2019). Language models are unsupervised multitask learners.
- Raffel, C. et al. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*.
- Raschka, S. (2024). *Build a Large Language Model (from Scratch)*. Manning.
- Vafa, K., Athey, S., and Blei, D.M. (2025). Estimating wage disparities using foundation models. *PNAS*, 122(22).
- Vaswani, A. et al. (2017). Attention is all you need. *NeurIPS*.
- Ziems, C. et al. (2024). Can large language models transform computational social science? *Computational Linguistics*.